

VERIQTA
ENGINEERING EXCELLENCE



20 PAGES
PRACTICAL KNOWLEDGE
REAL-WORLD READY



AWS

CLOUD ENGINEER

HANDBOOK

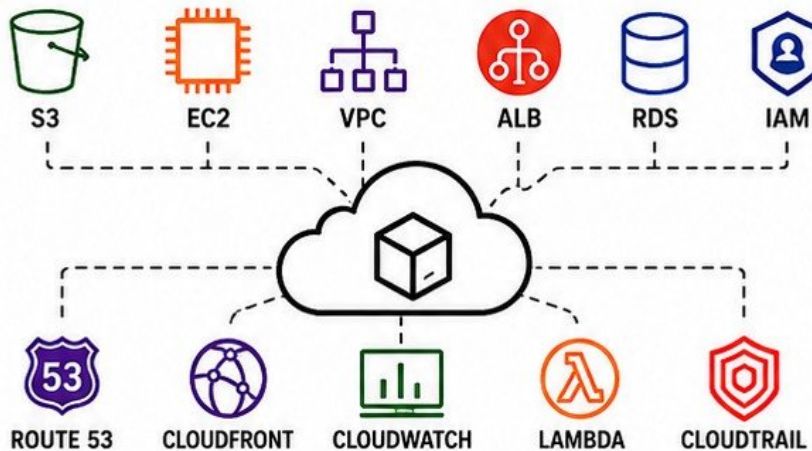
JUNIOR LEVEL

- ✓ REAL SCENARIOS
- ✓ REAL QUESTIONS
- ✓ REAL ANSWERS



Real-World Technical Questions, Scenarios,
and Production Fundamentals

- ✓ Technical Questions
- ✓ Real-World Scenarios
- ✓ Strong Answers
- ✓ Production Fundamentals
- ✓ Interview Ready



- ✓ AWS Fundamentals
- ✓ Troubleshooting Workflows
- ✓ Architecture Scenarios
- ✓ Operational Excellence
- ✓ Confidence Booster



PRACTICAL
Learn what
interviewers
actually test



STRUCTURED
20 focused
pages. One
topic per page



RELEVANT
Junior-level
content aligned
with real roles



RESULTS
Build knowledge.
Improve answers.
Get hired.



MASTER AWS. SOLVE REAL PROBLEMS. BUILD REAL SKILLS.

SHIP WITH CONFIDENCE.



VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



VERIQTA



VERIQTA

1. AWS INTERVIEW FOUNDATIONS



VERIQTA

Build the right mindset. Answer with confidence.

1 WHAT JUNIOR AWS INTERVIEWS ACTUALLY TEST

**UNDERSTANDING OF CORE AWS SERVICES**

How they work and when to use them

**PROBLEM SOLVING AND TROUBLESHOOTING**

Identify issues and fix them step by step

**ARCHITECTURAL FUNDAMENTALS**

Design simple, scalable, and secure solutions

**SECURITY AND BEST PRACTICES**

Apply AWS security principles and standards

**COMMUNICATION AND EXPLANATION**

Explain your thinking clearly and confidently

2 CLOUD FUNDAMENTALS VERSUS MEMORIZATION

VERIQTA

FOCUS ON THIS



- ✓ UNDERSTAND CONCEPTS
- ✓ KNOW WHY AND HOW
- ✓ APPLY TO REAL SCENARIOS
- ✓ THINK LIKE AN ENGINEER
- ✓ USE LOGIC AND COMMON SENSE

VERIQTA

NOT THIS



- ✗ MEMORIZE SERVICE LISTS
- ✗ RANDOM FACTS WITHOUT CONTEXT
- ✗ RECALL FEATURES WITHOUT USE CASES
- ✗ SAY WHAT YOU DON'T UNDERSTAND
- ✗ GUESS WITHOUT THINKING

VS

3 HOW TO APPROACH SCENARIO QUESTIONS

VERIQTA

**1. UNDERSTAND THE PROBLEM**

Listen carefully and identify the goal, constraints, and requirements.

**2. CLARIFY IF NEEDED**

Ask smart questions to remove ambiguity.

**3. BREAK IT DOWN**

Divide the problem into smaller parts.

**4. THINK OUT LOUD**

Explain your thought process step by step.

**5. PROPOSE A SOLUTION**

Share your approach, services, and reasoning.

**6. REVIEW AND IMPROVE**

Consider trade-offs, security, cost, and scalability.

4 EXPLAINING TECHNICAL DECISIONS CLEARLY

VERIQTA

**BE STRUCTURED**

Start with the overview, then go into details.

**JUSTIFY YOUR CHOICES**

Explain why you chose a service or approach.

**DISCUSS TRADE-OFFS**

Mention alternatives and their pros and cons.

**CONSIDER SECURITY AND RELIABILITY**

Show awareness of best practices.

**THINK ABOUT COST AND SCALABILITY**

Design for growth and efficiency.

**BE CONFIDENT BUT HONEST**

If you don't know, say it and keep thinking out loud.



INTERVIEW TIP:

Interviewers want to see how you think, not just what you know. Show your logic, stay calm, and communicate clearly.

VERIQTA

VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



VERIQTA



2. AWS GLOBAL INFRASTRUCTURE

VERIQTA

BUILDING BLOCKS OF A RELIABLE, SCALABLE, AND HIGHLY AVAILABLE CLOUD

1 REGIONS

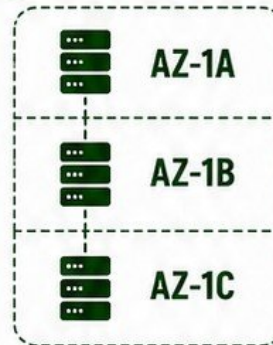
VERIQTA



- A REGION IS A GEOGRAPHIC AREA THAT CONTAINS MULTIPLE AVAILABILITY ZONES.
- EACH REGION IS ISOLATED FROM OTHERS.
- CHOOSE REGIONS CLOSE TO YOUR USERS FOR LOWER LATENCY.
- EXAMPLE: US EAST (N. VIRGINIA) – us-east-1

2 AVAILABILITY ZONES (AZs)

VERIQTA



- AN AZ IS ONE OR MORE DISCRETE DATA CENTERS WITHIN A REGION.
- DESIGNED TO BE PHYSICALLY SEPARATE FROM EACH OTHER.
- CONNECTED WITH LOW LATENCY, HIGH BANDWIDTH NETWORKS.
- BUILD FOR FAILURE ACROSS MULTIPLE AZs.

3 EDGE LOCATIONS

VERIQTA



- EDGE LOCATIONS ARE PART OF AMAZON CLOUD FRONT.
- THEY ARE DATA CENTERS AROUND THE WORLD.
- CONTENT IS CACHED CLOSER TO USERS.
- REDUCES LATENCY AND IMPROVES PERFORMANCE.
- USED FOR CDN AND GLOBAL CONTENT DELIVERY.

4 REGIONAL VS GLOBAL SERVICES

VERIQTA

REGIONAL SERVICES



- EXIST IN ONE REGION AT A TIME.
- RESOURCES ARE REGION-SPECIFIC.
- EXAMPLES: EC2, RDS, VPC, S3

GLOBAL SERVICES



- EXIST ACROSS ALL REGIONS.
- ONE RESOURCE IS AVAILABLE GLOBALLY.
- EXAMPLES: IAM, ROUTE 53, CLOUDFRONT, S3 (AS A SERVICE)

VS

5 HIGH-AVAILABILITY BASICS

VERIQTA



- DISTRIBUTE RESOURCES ACROSS MULTIPLE AZs.
- USE LOAD BALANCERS TO SPREAD TRAFFIC.
- ELIMINATE SINGLE POINTS OF FAILURE.
- AUTOMATIC FAILOVER WITH HEALTH CHECKS.
- BACK UP DATA AND TEST DISASTER RECOVERY.
- DESIGN FOR FAILURE, NOT JUST SUCCESS.

6 INTERVIEW SCENARIOS

VERIQTA

- Q1: WHAT IS THE DIFFERENCE BETWEEN A REGION AND AN AVAILABILITY ZONE?
- Q2: HOW WOULD YOU DESIGN A HIGHLY AVAILABLE WEB APPLICATION ON AWS?
- Q3: IF AN ENTIRE AZ GOES DOWN, HOW DOES YOUR APPLICATION STAY AVAILABLE?
- Q4: WHICH AWS SERVICES ARE GLOBAL AND WHY?
- Q5: WHERE WOULD YOU STORE STATIC CONTENT THAT USERS ACCESS GLOBALLY?



UNDERSTAND THE INFRASTRUCTURE. DESIGN FOR FAILURE. DELIVER RELIABILITY.

THAT'S WHAT ENGINEERS DO.



VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



3. IAM AND ACCESS MANAGEMENT



VERIQTA

THE RIGHT ACCESS TO THE RIGHT PEOPLE FOR THE RIGHT REASONS

1 USERS, GROUPS, AND ROLES

USERS



An identity for a person or application.

Has credentials (password or access keys).

GROUPS



A collection of users.

Permissions are assigned to the group.

ROLES



An identity that you can assume.

Used by AWS services or external users.

2 IAM POLICIES

VERIQTA



- JSON documents that define permissions.
- Attached to users, groups, or roles.
- Determine what actions are allowed or denied.
- Types: AWS Managed, Customer Managed, Inline Policies.

EXAMPLE POLICY (ALLOW READ-ONLY S3 ACCESS)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:ListBucket", "s3:GetObject"],
    "Resource": ["arn:aws:s3:::my-bucket",
      "arn:aws:s3:::my-bucket/*"]
  }]
}
```

3 LEAST PRIVILEGE

VERIQTA



- Grant the minimum permissions required to perform a task.
- Reduces security risks.
- Review and remove unnecessary permissions regularly.
- Use Managed Policies when possible.
- Avoid using AdministratorAccess unless absolutely necessary.

4 ACCESS KEYS



- Long-term credentials for programmatic access.
- Consist of Access Key ID and Secret Access Key.
- Store securely.
- Rotate regularly.
- Do NOT hardcode in applications.

5 TEMPORARY CREDENTIALS

VERIQTA



- Short-term and more secure.
- Obtained by assuming a role.
- Used by AWS services, CLI, SDKs.
- Automatically expire (15 minutes to 12 hours).
- No long-term secrets to manage.

6 COMMON PERMISSION-DENIED SCENARIOS AND HOW TO FIX THEM

VERIQTA

SCENARIO	POSSIBLE CAUSE	HOW TO FIX
 AccessDenied when using AWS CLI	User or role lacks permission for the action.	Check IAM policy. Add the required permission.
 AccessDenied to an S3 bucket	Bucket policy denies access or missing s3:* permissions.	Review bucket policy and IAM policy. Ensure correct permissions.
 Cannot assume an IAM role	Trust relationship does not allow the user/service.	Update the role trust policy to allow the principal.
 Access key not working	Key is inactive, deleted, or permissions are missing.	Create a new access key or enable the existing key. Attach proper policies.
 Denied when accessing a resource in another account	Cross-account permissions are not configured.	Check resource-based policy and role trust relationships.
 Permission works in console but not in CLI/SDK	Using different user/role or region.	Verify credentials, region, and permissions for that identity.



IAM BEST PRACTICES



USE LEAST PRIVILEGE



ENABLE MFA FOR ALL USERS



REVIEW ACCESS REGULARLY



MONITOR WITH CLOUDTRAIL

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



4. EC2 FUNDAMENTALS AND TROUBLESHOOTING

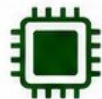
LAUNCH. CONFIGURE. CONNECT. TROUBLESHOOT.

1 INSTANCES AND AMIS



- AN INSTANCE IS A VIRTUAL SERVER IN THE CLOUD.
- AN AMI (AMAZON MACHINE IMAGE) IS A TEMPLATE USED TO LAUNCH INSTANCES.
- AMIS CAN BE AWS PROVIDED, COMMUNITY, OR CUSTOM.
- YOU CAN LAUNCH, STOP, START, AND TERMINATE INSTANCES.

2 INSTANCE TYPES



- DIFFERENT TYPES ARE OPTIMIZED FOR COMPUTE, MEMORY, STORAGE, OR NETWORK.
- EXAMPLES:

TYPE	BEST FOR
t3.micro	Burstable, Low Cost
m6i	General Purpose
c6i	Compute Intensive
r6i	Memory Intensive
i4i	Storage Intensive

3 KEY PAIRS



- USED FOR SSH ACCESS (LINUX INSTANCES).
- CREATE A KEY PAIR (.PEM FILE).
- DOWNLOAD AND STORE IT SECURELY.
- REQUIRED AT LAUNCH TIME.

4 SECURITY GROUPS



- ACT AS A VIRTUAL FIREWALL FOR YOUR INSTANCE.
- STATEFUL: ALLOW RULES AUTOMATICALLY ALLOW RETURN TRAFFIC.
- CONTROL INBOUND AND OUTBOUND TRAFFIC.
- EXAMPLE: ALLOW SSH (PORT 22) FROM YOUR IP ONLY.

5 EBS (ELASTIC BLOCK STORE)



- PROVIDES PERSISTENT STORAGE FOR EC2 INSTANCES.
- TYPES: gp3, gp2, io1, io2, st1, sc1.
- SNAPSHOTS BACK UP YOUR VOLUMES.
- VOLUMES CAN BE ATTACHED TO ONE INSTANCE AT A TIME.

6 INSTANCE LIFECYCLE



7 WHY CAN'T I CONNECT TO MY EC2 INSTANCE?

POSSIBLE CAUSE	WHAT TO CHECK	HOW TO FIX
SECURITY GROUP BLOCKING	CHECK INBOUND RULES FOR SSH (PORT 22)	ADD INBOUND RULE TO ALLOW YOUR IP ON PORT 22
NETWORK ACL BLOCKING	CHECK NACL INBOUND/OUTBOUND RULES	ADD RULE TO ALLOW TRAFFIC ON PORT 22
INCORRECT IP ADDRESS	VERIFY PUBLIC IP OR DNS NAME	USE THE CORRECT PUBLIC IP OR DNS
WRONG KEY PAIR	CHECK THE KEY USED TO CONNECT	USE THE CORRECT .PEM FILE
INSTANCE NOT RUNNING	CHECK INSTANCE STATE	START THE INSTANCE
OS FIREWALL BLOCKING	CHECK OS FIREWALL (UFW, FIREWALLD, ETC.)	ALLOW SSH (PORT 22) IN OS FIREWALL
WRONG USERNAME	USE THE CORRECT DEFAULT USER	EXAMPLE: UBUNTU (ubuntu), AMAZON LINUX (ec2-user)
ROUTE TABLE / IGW ISSUE	CHECK INTERNET GATEWAY AND ROUTES	ATTACH IGW AND CHECK ROUTING



CHECK. VERIFY. UNDERSTAND. SOLVE.
THAT'S HOW ENGINEERS THINK.



VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



5. AMAZON VPC FUNDAMENTALS



BUILD SECURE, SCALABLE, AND WELL-ARCHITECTED NETWORKS IN AWS

1 VPCS



- A VPC IS A VIRTUAL NETWORK ISOLATED IN THE AWS CLOUD.
- YOU HAVE COMPLETE CONTROL OVER IP ADDRESS RANGE, SUBNETS, ROUTES, AND SECURITY.
- EACH VPC IS LAUNCHED IN A REGION.

2 CIDR BLOCKS

EXAMPLE CIDR: 10.0.0.0/16



- CIDR (CLASSLESS INTER-DOMAIN ROUTING) DEFINES THE IP RANGE.
- CHOOSE A RANGE THAT DOES NOT OVERLAP WITH OTHER NETWORKS.
- USE /16 FOR VPC, /24 FOR SUBNETS.

3 PUBLIC AND PRIVATE SUBNETS

PUBLIC SUBNET



- HAS A ROUTE TO THE INTERNET (VIA IGW).
- CAN BE ACCESSED FROM THE INTERNET.
- USE FOR: ALB, BASTION HOSTS, NAT GATEWAYS.

PRIVATE SUBNET



- NO DIRECT ROUTE TO THE INTERNET.
- NOT ACCESSED FROM THE INTERNET.
- USE FOR: APP SERVERS, DATABASES, INTERNAL SERVICES.

4 ROUTE TABLES



- CONTROLS WHERE NETWORK TRAFFIC IS SENT.
- EACH SUBNET IS ASSOCIATED WITH A ROUTE TABLE.
- MAIN ROUTE TABLE IS CREATED BY DEFAULT.
- CUSTOM ROUTE TABLES CAN BE CREATED.

COMMON ROUTES

DESTINATION	TARGET
10.0.0.0/16	local
0.0.0.0/0	igw-xxxxxx (Internet Gateway)
0.0.0.0/0	nat-xxxxxx (NAT Gateway)
172.16.0.0/16	vgn-xxxxxx (VPN Gateway)
pl-xxxxxx	vpce-xxxxxx (VPC Endpoint)

5 INTERNET GATEWAYS



- ATTACHED TO YOUR VPC.
- ENABLES COMMUNICATION BETWEEN YOUR VPC AND THE INTERNET.
- REQUIRED FOR PUBLIC SUBNETS.
- ONLY WORKS WITH IPV4.



6 NAT GATEWAYS



- ALLOWS INSTANCES IN PRIVATE SUBNETS TO ACCESS THE INTERNET FOR OUTBOUND TRAFFIC.
- DOES NOT ALLOW INBOUND TRAFFIC FROM THE INTERNET.
- DEPLOY IN A PUBLIC SUBNET.
- USE ELASTIC IP ADDRESS.

7 NETWORK INTERVIEW SCENARIOS

SCENARIO	QUESTION	WHAT THEY EXPECT YOU TO CONSIDER
 WEB SERVER NOT ACCESSIBLE FROM INTERNET	YOUR EC2 INSTANCE IS RUNNING, BUT YOU CAN'T ACCESS THE WEBSITE.	CHECK IGW ATTACHED, PUBLIC IP, SECURITY GROUP INBOUND RULE, ROUTE TABLE (0.0.0.0/0 → IGW), NACL.
 PRIVATE INSTANCES CAN'T ACCESS INTERNET	INSTANCES IN PRIVATE SUBNET CAN'T INSTALL UPDATES.	CHECK NAT GATEWAY, ROUTE TABLE (0.0.0.0/0 → NAT), SUBNET ASSOCIATION.
 DATABASE ACCESS ISSUES	APP SERVER CAN'T CONNECT TO DATABASE IN ANOTHER SUBNET.	CHECK SECURITY GROUP RULES, NACL, ROUTE TABLE, SUBNET CIDR, NETWORK ACL STATELESS RULES.
 VPC PEERING / CONNECTIVITY	TWO VPCS CAN'T COMMUNICATE EVEN AFTER PEERING.	CHECK ROUTE TABLES IN BOTH VPCS, PEERING CONNECTION STATUS, SECURITY GROUPS.
 OVERLAPPING CIDR	VPC PEERING FAILED DUE TO OVERLAPPING CIDR.	CIDR RANGES MUST NOT OVERLAP. REDESIGN IP ADDRESSING.



UNDERSTAND THE NETWORK. CONTROL THE ROUTES. SECURE THE ACCESS.
THAT'S THE BASIS OF EVERY CLOUD ARCHITECTURE.



VERIQTA SOCIAL



6. SECURITY GROUPS VS NETWORK ACLs

CONTROL TRAFFIC. APPLY THE RIGHT RULES. SECURE YOUR VPC.

1 STATEFUL VS STATELESS FILTERING

SECURITY GROUPS (STATEFUL)



- STATEFUL: IF YOU ALLOW INBOUND TRAFFIC, THE RETURN TRAFFIC IS AUTOMATICALLY ALLOWED.
- YOU ONLY NEED TO RULE INBOUND TRAFFIC.

NETWORK ACLs (STATELESS)



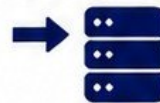
- STATELESS: EACH DIRECTION IS EVALUATED SEPARATELY.
- YOU MUST EXPLICITLY ALLOW BOTH INBOUND AND OUTBOUND TRAFFIC.

2 INBOUND AND OUTBOUND RULES

SECURITY GROUPS

INBOUND RULES

Define which traffic is allowed to your resources.



OUTBOUND RULES

Define which traffic your resources can initiate.



NETWORK ACLs

INBOUND RULES

Define which traffic is allowed into the subnet.



OUTBOUND RULES

Define which traffic is allowed out of the subnet.



3 RULE EVALUATION

SECURITY GROUPS

- 1 All rules are evaluated.
- 2 If a rule allows, traffic is allowed.
- 3 If no rule allows, traffic is denied (implicit deny).

NETWORK ACLs

- 1 Rules are evaluated in order by rule number (lowest first).
- 2 First match wins (allow or deny).
- 3 If no rule matches, traffic is denied (implicit deny).

4 TROUBLESHOOTING BLOCKED TRAFFIC

- ✓ CHECK SECURITY GROUP INBOUND RULES.
- ✓ CHECK NACL INBOUND RULES ON THE SUBNET.
- ✓ CHECK NACL OUTBOUND RULES.
- ✓ CHECK SECURITY GROUP OUTBOUND RULES.
- ✓ CHECK ROUTE TABLE.
- ✓ CHECK NACL ASSOCIATION.
- ✓ CHECK SERVER / APPLICATION / FIREWALL.

5 SCENARIO-BASED QUESTIONS

- Q1 An instance cannot be reached from the internet. What do you check first?
- Q2 You allowed inbound rule in the security group, but still no response. What else could be blocking the traffic?
- Q3 A private subnet instance cannot access the internet. What is the likely issue?
- Q4 Traffic works one way but not the other. What could be the cause?
- Q5 How would you allow SSH access only from a specific IP?
- Q6 How do you deny all traffic by default using NACLs?

6 QUICK COMPARISON

FEATURE	SECURITY GROUPS	NETWORK ACLs
TYPE	STATEFUL	STATELESS
SCOPE	ATTACHED TO ENI (INSTANCE LEVEL)	ATTACHED TO SUBNET
DEFAULT RULE	ALLOW ALL OUTBOUND	DENY ALL INBOUND AND OUTBOUND
RULE EVALUATION	ALL RULES EVALUATED	FIRST MATCH WINS (BY RULE NUMBER)
RULES REQUIRED	INBOUND ONLY (OUTBOUND OPTIONAL)	BOTH INBOUND AND OUTBOUND
USE CASE	CONTROL TRAFFIC TO / FROM INSTANCE	CONTROL TRAFFIC AT SUBNET LEVEL



COMBINE SECURITY GROUPS AND NACLs FOR DEFENSE IN DEPTH AND **NETWORK CONTROL**.



VERIQTA SOCIAL



7. AMAZON S3

STORE ANYTHING. ACCESS ANYWHERE. SECURE EVERYTHING.



1 BUCKETS AND OBJECTS



- BUCKET: CONTAINER FOR STORING OBJECTS.
- BUCKET NAMES ARE GLOBALLY UNIQUE.
- OBJECTS: THE DATA YOU STORE (FILE + METADATA).
- OBJECTS ARE STORED IN A BUCKET.
- KEY = FULL PATH TO THE OBJECT.



2 STORAGE CLASSES



S3 STANDARD

Frequent access.
Millisecond access.



S3 STANDARD-IA

Infrequent access.
Lower cost.



S3 ONE ZONE-IA

Infrequent access.
One AZ.
Lower cost.



S3 GLACIER INSTANT RETRIEVAL

Archival data.
Millisecond access.



S3 GLACIER FLEXIBLE RETRIEVAL

Minutes to hours
to retrieve.



S3 GLACIER DEEP ARCHIVE

Lowest cost.
Hours to retrieve.

3 VERSIONING



- KEEPS MULTIPLE VERSIONS OF AN OBJECT.
- PROTECTS AGAINST ACCIDENTAL DELETION AND OVERWRITES.
- ENABLED AT THE BUCKET LEVEL.
- SUSPEND DOES NOT DELETE EXISTING VERSIONS.

4 LIFECYCLE POLICIES



- AUTOMATICALLY MOVE OR DELETE OBJECTS BASED ON RULES.
- USE CASES:
 - ✓ MOVE TO CHEAPER STORAGE AFTER X DAYS.
 - ✓ DELETE OLD OBJECTS.
 - ✓ REDUCE COSTS.
- APPLIED AT THE BUCKET LEVEL.

5 BUCKET POLICIES



- JSON-BASED POLICIES THAT CONTROL ACCESS TO THE BUCKET AND OBJECTS.
- CAN GRANT OR DENY ACCESS.
- USE PRINCIPALS: AWS ACCOUNTS, IAM USERS, ROLES.
- EVALUATED BEFORE OBJECT ACCESS.

6 ENCRYPTION



- SERVER-SIDE ENCRYPTION (SSE-S3)
- SSE-KMS (AWS KMS KEYS)
- SSE-C (CUSTOMER PROVIDED KEYS)
- CLIENT-SIDE ENCRYPTION
- ENCRYPTION SHOULD BE ENABLED BY DEFAULT.

7 PUBLIC-ACCESS TROUBLESHOOTING

CHECK	WHAT TO LOOK FOR	HOW TO FIX
1  BUCKET POLICY	LOOK FOR "ALLOW" TO "*" OR PUBLIC PRINCIPALS.	REMOVE PUBLIC ALLOW STATEMENTS.
2  PUBLIC ACCESS BLOCK	CHECK ACCOUNT AND BUCKET PUBLIC ACCESS BLOCK SETTINGS.	ENABLE BLOCK ALL PUBLIC ACCESS.
3  IAM POLICIES	CHECK IF IAM POLICY ALLOWS PUBLIC ACCESS.	REMOVE UNNECESSARY PUBLIC PERMISSIONS.
4  OBJECT ACL	CHECK OBJECT ACL FOR PUBLIC READ.	REMOVE PUBLIC ACLS.
5  BUCKET ACL	CHECK BUCKET ACL FOR PUBLIC ACCESS.	REMOVE PUBLIC ACLS.
6  TEST ACCESS	TEST IN BROWSER OR AWS CLI.	VERIFY AGAIN AFTER CHANGES.



SECURE YOUR DATA. MANAGE YOUR COSTS.

S3 DONE RIGHT IS SIMPLE AND POWERFUL.



VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



8.

AWS LOAD BALANCING

DISTRIBUTE TRAFFIC. IMPROVE AVAILABILITY. SCALE WITH CONFIDENCE.

1 APPLICATION LOAD BALANCER (ALB)



- WORKS AT LAYER 7 (HTTP/HTTPS).
- ROUTES TRAFFIC BASED ON CONTENT (HOST, PATH, HEADER).
- IDEAL FOR WEB APPLICATIONS.
- SUPPORTS TLS TERMINATION.
- SUPPORTS REDIRECTS AND URL REWRITES.

2 NETWORK LOAD BALANCER (NLB)



- WORKS AT LAYER 4 (TCP/UDP).
- HIGH PERFORMANCE AND LOW LATENCY.
- HANDLES MILLIONS OF REQUESTS PER SECOND.
- IDEAL FOR NON-HTTP APPLICATIONS.
- SUPPORTS STATIC IP (ELASTIC IP).

3 LISTENERS



- CHECK FOR CONNECTIONS ON A PORT AND PROTOCOL.
- FORWARD REQUESTS TO TARGET GROUPS.
- ALB: HTTP, HTTPS.
- NLB: TCP, TLS, UDP, TCP_UDP.
- DEFAULT ACTION IS TO FORWARD OR RETURN A RESPONSE.

4 TARGET GROUPS



- GROUP OF TARGETS (EC2, IP, LAMBDA) THAT RECEIVE TRAFFIC.
- EACH TARGET GROUP HAS A HEALTH CHECK.
- LOAD BALANCER ROUTES TRAFFIC ONLY TO HEALTHY TARGETS.
- SUPPORTS MULTIPLE TARGET TYPES.

5 HEALTH CHECKS



- LB CHECKS TARGET HEALTH AT REGULAR INTERVALS.
- CONFIGURE PROTOCOL, PATH, PORT, AND THRESHOLDS.
- ONLY HEALTHY TARGETS RECEIVE TRAFFIC.
- UNHEALTHY TARGETS ARE REMOVED AUTOMATICALLY.

6 UNHEALTHY TARGETS



- CAN BE CAUSED BY APP ERRORS, TIMEOUTS, OR MISCONFIGURATION.
- CHECK TARGET HEALTH STATUS IN THE CONSOLE OR CLI.
- UNHEALTHY TARGETS DO NOT RECEIVE TRAFFIC.
- FIX THE ROOT CAUSE AND WAIT FOR TARGET TO RECOVER.

7 502, 503, AND TIMEOUT SCENARIOS

ERROR	WHAT IT MEANS	COMMON CAUSES	HOW TO FIX
502 BAD GATEWAY	The load balancer received an invalid response from the target.	• Target application is down or crashed. • Application returned a malformed response. • Connection reset by the target. • Security group or NACL blocking traffic.	• Check target health status. • Review application logs. • Verify security group and NACL rules. • Ensure target is listening on the correct port.
503 SERVICE UNAVAILABLE	The load balancer cannot route the request to any healthy target.	• No healthy targets in the target group. • All targets are down or failing health checks. • Incorrect health check configuration. • Targets in different AZ and cross-zone disabled.	• Check target group health. • Fix application or infrastructure issues. • Verify health check settings. • Enable cross-zone load balancing.
TIMEOUT (504)	The load balancer could not get a response from the target in time.	• Application is slow. • Backend processing takes too long. • Health check timeout too low. • Network latency or congestion.	• Optimize application performance. • Increase idle timeout. • Adjust health check timeouts. • Check network connectivity.

LOAD BALANCING BEST PRACTICES



USE HEALTH CHECKS TO ENSURE ONLY HEALTHY TARGETS RECEIVE TRAFFIC.



DISTRIBUTE TRAFFIC ACROSS MULTIPLE AZs FOR HIGH AVAILABILITY.



ENABLE TLS TERMINATION FOR SECURITY AND PERFORMANCE.



MONITOR METRICS AND LOGS REGULARLY.



USE APPROPRIATE TIMEOUT SETTINGS TO MATCH YOUR APPLICATION.

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



9. AUTO SCALING

SCALE AUTOMATICALLY. STAY AVAILABLE.



1 AUTO SCALING GROUPS (ASG)



- MANAGES A FLEET OF IDENTICAL EC2 INSTANCES.
- ENSURES HIGH AVAILABILITY AND ELASTICITY.
- REPLACES UNHEALTHY INSTANCES AUTOMATICALLY.
- WORKS WITH LOAD BALANCERS.

2 CAPACITY SETTINGS



MINIMUM CAPACITY
The minimum number of instances in the group.



DESIRED CAPACITY
The target number of instances to maintain.



MAXIMUM CAPACITY
The maximum number of instances allowed.

ASG CAPACITY RANGE
 $\text{MIN} \leq \text{DESIRED} \leq \text{MAX}$

3 SCALING POLICIES



TARGET TRACKING

- MAINTAINS A TARGET VALUE (CPU, REQUESTS, ETC).
- AUTOMATICALLY ADJUSTS CAPACITY.



STEP SCALING

- ADD OR REMOVE A FIXED NUMBER OF INSTANCES.
- BASED ON CLOUDWATCH ALARMS.

USES AMAZON CLOUDWATCH ALARMS TO TRIGGER SCALING ACTIONS.

4 HEALTH CHECKS



- ASG MONITORS INSTANCE HEALTH USING:
 - ✓ EC2 STATUS CHECKS
 - ✓ ELB HEALTH CHECKS (IF ATTACHED)
- UNHEALTHY INSTANCES ARE TERMINATED AND REPLACED.
- ENSURES ONLY HEALTHY INSTANCES RECEIVE TRAFFIC.

5 LAUNCH TEMPLATES



- DEFINES HOW INSTANCES ARE LAUNCHED.
- INCLUDES AMI, INSTANCE TYPE, KEY PAIR, SECURITY GROUPS, USER DATA, ETC.
- RECOMMENDED OVER LAUNCH CONFIGURATIONS.
- SUPPORTS VERSIONING.

6 HOW AUTO SCALING WORKS



METRIC
MONITORED



ALARM
TRIGGERED



SCALING
ACTION



CAPACITY
ADJUSTED

MONITOR → ALARM → ACTION → SCALE

7 SCALING TROUBLESHOOTING SCENARIOS

SCENARIO	POSSIBLE CAUSES	WHAT TO CHECK	HOW TO FIX
 ASG DOES NOT SCALE OUT	• ALARM NOT TRIGGERING • MAX CAPACITY REACHED • SCALING POLICY MISCONFIGURED • CLOUDWATCH METRIC ISSUE	• CHECK CLOUDWATCH ALARMS • CHECK ASG CAPACITY SETTINGS • CHECK SCALING POLICY DETAILS • VERIFY METRICS ARE PUBLISHED	• FIX ALARM THRESHOLD OR METRIC • INCREASE MAX CAPACITY • CORRECT SCALING POLICY • PUBLISH CORRECT METRICS
 ASG DOES NOT SCALE IN	• ALARM NOT TRIGGERING • MIN CAPACITY LOCKING • COOLDOWN PERIOD ACTIVE • METRIC NOT DROPPING	• CHECK CLOUDWATCH ALARMS • CHECK MIN CAPACITY • CHECK INSTANCE COOLDOWN • VERIFY METRIC TREND	• ADJUST ALARM THRESHOLD • LOWER MIN CAPACITY (IF NEEDED) • WAIT FOR COOLDOWN TO FINISH • VERIFY WORKLOAD IS LOWER
 INSTANCES KEEP TERMINATING	• FAILING HEALTH CHECKS • APP OR CONFIGURATION ERROR • UNSTABLE APPLICATION • ELB HEALTH CHECK FAILING	• CHECK ASG ACTIVITY HISTORY • CHECK INSTANCE STATUS CHECKS • CHECK ELB TARGET HEALTH • CHECK APPLICATION LOGS	• FIX APPLICATION OR CONFIGURATION • CORRECT SECURITY GROUP / NACL • IMPROVE APP STABILITY • FIX HEALTH CHECK PATH OR PORT
 NEW INSTANCES NOT LAUNCHING	• QUOTA LIMIT REACHED • LAUNCH TEMPLATE ERROR • SUBNET IP ADDRESS EXHAUSTION • IAM PERMISSION ISSUE	• CHECK SERVICE QUOTAS • VERIFY LAUNCH TEMPLATE • CHECK SUBNET IP AVAILABILITY • CHECK IAM ROLE PERMISSIONS	• REQUEST QUOTA INCREASE • CORRECT LAUNCH TEMPLATE • USE LARGER SUBNET / MORE SUBNETS • FIX IAM ROLE PERMISSIONS

AUTO SCALING BEST PRACTICES



SET APPROPRIATE MIN, DESIRED, AND MAX CAPACITY.



USE TARGET TRACKING FOR BEST RESULTS.



USE HEALTH CHECKS TO ENSURE RELIABLE SCALING.



USE LAUNCH TEMPLATES WITH BEST PRACTICES.



MONITOR METRICS AND ADJUST REGULARLY.

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



10. AWS DATABASES



STORE. MANAGE. PROTECT. SCALE YOUR DATA WITH AWS.

1 AMAZON RDS



- MANAGED RELATIONAL DATABASE SERVICE.
- SUPPORTS ENGINES: MYSQL, POSTGRESQL, MARIADB, ORACLE, SQL SERVER, ETC.
- AUTOMATED BACKUPS, PATCHING, AND MONITORING.
- EASY TO SET UP AND SCALE.
- SUPPORTS MULTI-AZ DEPLOYMENT.

2 AMAZON AURORA



- AWS-NATIVE RELATIONAL DATABASE.
- COMPATIBLE WITH MYSQL AND POSTGRESQL.
- UP TO 5X BETTER PERFORMANCE THAN MYSQL.
- BUILT-IN HIGH AVAILABILITY AND AUTO SCALING.
- STORAGE AUTO-GROWS INCREMENTS OF 10 GB.

3 AMAZON DYNAMODB



- FULLY MANAGED NO SQL DATABASE.
- KEY-VALUE AND DOCUMENT DATABASE.
- SINGLE-DIGIT MILLISECOND PERFORMANCE AT ANY SCALE.
- AUTOMATIC SCALING AND HIGH AVAILABILITY.
- IDEAL FOR HIGH-TRAFFIC AND SERVERLESS APPS.

4 RELATIONAL VS NOSQL

RELATIONAL (SQL)



- STRUCTURED DATA
- SCHEMA-BASED
- JOINS AND ACID TRANSACTIONS
- EXAMPLES: RDS, AURORA

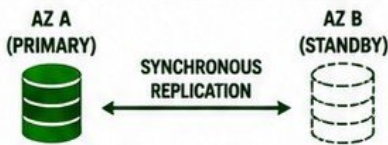
NOSQL



- FLEXIBLE DATA MODELS
- SCHEMA-LESS
- HORIZONTAL SCALING
- EXAMPLES: DYNAMODB

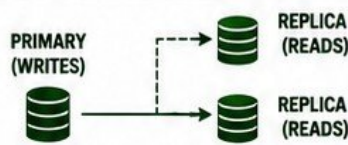
5 HIGH AVAILABILITY: MULTI-AZ AND READ REPLICAS

MULTI-AZ (HIGH AVAILABILITY)



- STANDBY DB IN A DIFFERENT AZ.
- AUTOMATIC FAILOVER IF PRIMARY FAILS.
- MINIMAL DOWNTIME.
- USED FOR RDS AND AURORA.

READ REPLICAS (SCALABILITY)



- REPLICAS HANDLING READ TRAFFIC.
- IMPROVES READ PERFORMANCE.
- ASYNCHRONOUS REPLICATION.
- SUPPORTED BY RDS AND AURORA.

AURORA CLUSTER ARCHITECTURE



- SHARED CLUSTER STORAGE.
- HIGHLY AVAILABLE AND DURABLE.
- READ SCALING WITH MULTIPLE ENDPOINTS.

6 CHOOSING THE RIGHT DATABASE

USE CASE	BEST CHOICE	WHY	KEY FEATURES
TRADITIONAL WEB APPS	AMAZON RDS	SIMPLE, COST-EFFECTIVE, EASY TO MANAGE	ACID COMPLIANCE, MULTI-AZ, BACKUPS, MAINTENANCE
HIGH PERFORMANCE APPS	AMAZON AURORA	HIGH PERFORMANCE, AUTO SCALING, COST EFFICIENT	5X MYSQL PERFORMANCE, MULTI-AZ, AUTO SCALING
MOBILE APPS / GAMING	DYNAMODB	MASSIVE SCALE, LOW LATENCY, SERVERLESS FRIENDLY	SINGLE-DIGIT MS LATENCY, AUTO SCALING, PAY PER REQUEST
ANALYTICS / REPORTING	AURORA + READ REPLICAS	POWERFUL READ SCALING FOR ANALYTICS	MULTIPLE READ REPLICAS, FAST QUERIES
SESSION STORE / CACHING	DYNAMODB	LOW LATENCY, HIGH THROUGHPUT, SCHEMALESS	KEY-VALUE STORE, TTL, ON-DEMAND SCALING

7 DATABASE TROUBLESHOOTING SCENARIOS

SCENARIO	POSSIBLE CAUSES	WHAT TO CHECK	HOW TO FIX
CAN'T CONNECT TO RDS / AURORA	SECURITY GROUP, NACL, IAM, DB INSTANCE DOWN	• SECURITY GROUP INBOUND RULES • DATABASE STATUS • VPC, SUBNET, NACL	• ALLOW PORT (3306 / 5432 ETC.) • START DB INSTANCE • CHECK NETWORK CONNECTIVITY
HIGH CPU / PERFORMANCE ISSUES	INSUFFICIENT CAPACITY, SLOW QUERIES	• CLOUDWATCH METRICS • SLOW QUERY LOGS • DB INSTANCE CLASS	• UPGRADE INSTANCE CLASS • OPTIMIZE QUERIES • ADD READ REPLICAS
FAILOVER OCCURRED	PRIMARY INSTANCE FAILURE, STORAGE OR NETWORK ISSUE	• EVENTS IN RDS CONSOLE • CLOUDWATCH ALARMS	• VERIFY STANDBY HEALTH • CHECK APPLICATION CONNECTIONS
DYNAMODB THROTTLING	EXCEEDED PROVISIONED CAPACITY	• CLOUDWATCH THROTTLING REQUESTS • READ / WRITE CAPACITY UNITS	• INCREASE CAPACITY UNITS • ENABLE AUTO SCALING

DATABASE BEST PRACTICES



ENABLE ENCRYPTION AT REST AND IN TRANSIT.



AUTOMATE BACKUPS AND TEST RESTORES REGULARLY.



USE LEAST PRIVILEGE IAM POLICIES.



MONITOR WITH CLOUDWATCH AND SET ALARMS.



KEEP YOUR DATABASE ENGINES AND PATCHES UPDATED.

VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta

VERIQTA

11. DNS AND ROUTE 53

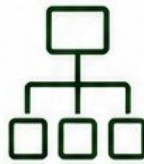
RELIABLE DNS. GLOBAL AVAILABILITY. HIGH PERFORMANCE.

1 DNS FUNDAMENTALS



- DNS (DOMAIN NAME SYSTEM) TRANSLATES HUMAN-READABLE NAMES TO IP ADDRESSES.
- DNS IS HIERARCHICAL AND DISTRIBUTED.
- IT WORKS ON A CLIENT-RESOLVER-NAMESERVER MODEL.
- DNS USES UDP 53 FOR QUERIES AND TCP 53 FOR ZONE TRANSFERS.

2 HOSTED ZONES



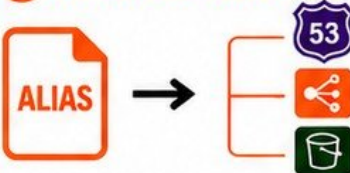
- A HOSTED ZONE IS A CONTAINER FOR DNS RECORDS.
- PUBLIC HOSTED ZONES RESOLVE NAMES ON THE INTERNET.
- PRIVATE HOSTED ZONES RESOLVE NAMES WITHIN A VPC.
- YOU CAN HAVE ONE PUBLIC AND MULTIPLE PRIVATE HOSTED ZONES.

3 RECORD TYPES



- **A** • MAPS DOMAIN TO IPV4 ADDRESS.
- **AAAA** • MAPS DOMAIN TO IPV6 ADDRESS.
- **CNAME** • ALIASES ONE DOMAIN TO ANOTHER DOMAIN.
- **MX** • MAIL EXCHANGE RECORD.
- **TXT** • TEXT RECORD FOR VERIFICATION, SPF, DKIM, ETC.
- **NS** • NAMESERVER RECORD.
- **SRV** • SERVICE RECORD FOR SPECIFYING SERVICES.

4 ALIAS RECORDS



- ALIAS RECORDS ALLOW YOU TO POINT A ROOT OR SUBDOMAIN TO AWS RESOURCES.
- WORKS WITH: ELB, CLOUDFRONT, S3, API GATEWAY, VPC ENDPOINTS, AND MORE.
- ALIAS RECORDS ARE AUTOMATICALLY HEALTH CHECKED (WHEN APPLICABLE).
- THEY DO NOT INCUR ANY ADDITIONAL QUERY CHARGES.

5 ROUTING POLICIES



- SIMPLE** DEFAULT POLICY. ROUTES TRAFFIC TO A SINGLE RESOURCE.
- WEIGHTED** ROUTES TRAFFIC BASED ON ASSIGNED WEIGHTS.
- LATENCY** ROUTES USERS TO THE REGION WITH THE LOWEST LATENCY.
- FAILOVER** ROUTES TO ACTIVE RESOURCE. FAILS OVER TO STANDBY.
- GEOLOCATION** ROUTES TRAFFIC BASED ON THE USER'S GEOGRAPHIC LOCATION.
- GEOPROXIMITY** ROUTES TRAFFIC BASED ON RESOURCE PROXIMITY TO USERS.

6 TTL (TIME TO LIVE)



- TTL DEFINES HOW LONG DNS RECORDS ARE CACHED BY RESOLVERS.
- LOWER TTL = FASTER UPDATES, MORE QUERIES.
- HIGHER TTL = SLOWER UPDATES, LESS QUERIES.
- COMMON TTL VALUES: 30, 60, 300, 900, 3600 (1 MIN TO 1 HOUR)
- PLAN TTL BASED ON FREQUENCY OF CHANGES AND TRAFFIC.

7 TROUBLESHOOTING SCENARIO: "THE APPLICATION WORKS BY IP BUT NOT BY DOMAIN NAME."

STEP	WHAT TO CHECK	HOW TO CHECK	WHAT TO LOOK FOR	HOW TO FIX
1	DNS RESOLUTION	nslookup your-domain.com or dig your-domain.com	• Returns correct IP address? • Any NXDOMAIN or timeout?	• Fix DNS records • Check hosted zone
2	NAMESERVERS	nslookup -type=ns your-domain.com	• Correct NS records? • Are they Route 53 NS?	• Update registrar NS • Fix delegation
3	A / ALIAS RECORD	dig your-domain.com +short	• Does it return the expected IP? • Is Alias pointing to the right resource?	• Create or update record • Verify Alias target
4	HEALTH CHECK (IF APPLICABLE)	Check Route 53 Health Checks in console	• Is the target healthy? • Any failed health checks?	• Fix target endpoint • Update health check settings
5	ROUTING POLICY	Review routing policy configuration	• Is traffic routed to the correct resource?	• Adjust policy settings • Verify weights / regions
6	TTL AND CACHING	Check TTL value and use whatsmydns.net	• Is old IP cached? • Are different locations showing old data?	• Lower TTL and wait • Flush local DNS cache
7	FIREWALL / SECURITY	Test IP in browser / ping Test domain in browser / ping	• Is the domain blocked? • WAF / SG / Firewall issues?	• Update firewall rules • Allow domain traffic
8	CLIENT / LOCAL CACHE	ipconfig /flushdns (Windows) sudo dscacheutil -flushcache (macOS)	• Local DNS cache causing issue?	• Flush DNS cache • Restart browser

ROUTE 53 BEST PRACTICES



USE ALIAS RECORDS FOR AWS RESOURCES



MONITOR HEALTH CHECKS REGULARLY



USE APPROPRIATE TTL VALUES



SECURE DNS WITH DNSSEC



ENABLE LOGGING FOR INSIGHTS

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



12. CLOUDFRONT AND CONTENT DELIVERY

GLOBAL PERFORMANCE. LOW LATENCY. HIGH AVAILABILITY.



1 CDN FUNDAMENTALS



- A CDN (CONTENT DELIVERY NETWORK) STORES CONTENT IN EDGE LOCATIONS (POINT OF PRESENCE).
- DELIVERS CONTENT CLOSER TO USERS FOR FASTER LOAD TIMES.
- REDUCES LATENCY AND IMPROVES USER EXPERIENCE.
- OFFLOADS TRAFFIC FROM ORIGIN SERVERS.

2 ORIGINS



- ORIGINS ARE THE SOURCE OF CONTENT FOR CLOUDFRONT.
- SUPPORTED ORIGINS: S3 BUCKETS, CUSTOM ORIGINS (HTTP/HTTPS), ALB, API GATEWAY, ETC.
- ORIGIN GROUPS PROVIDE FAILOVER BETWEEN MULTIPLE ORIGINS.

3 EDGE CACHING



- CONTENT IS CACHED AT EDGE LOCATIONS CLOSE TO USERS.
- REDUCES LOAD ON ORIGIN SERVERS.
- IMPROVES RESPONSE TIME AND SCALABILITY.
- CACHE HIT = FASTER DELIVERY AND LOWER COST.

4 CACHE BEHAVIOR



- DEFINES HOW CONTENT IS CACHED AND DELIVERED.

- KEY SETTINGS:

- ✓ ALLOWED HTTP METHODS
- ✓ CACHED HTTP METHODS
- ✓ CACHE POLICY
- ✓ ORIGIN REQUEST POLICY
- ✓ RESPONSE HEADERS POLICY
- ✓ TTL (TIME TO LIVE)
- ✓ COMPRESSION
- ✓ QUERY STRING FORWARDING
- ✓ COOKIES FORWARDING

TTL EXAMPLE

MIN TTL: 0 SECONDS

DEFAULT TTL: 86400 SECONDS (1 DAY)

MAX TTL: 31536000 SECONDS (1 YEAR)

Adjust based on how often content changes.

5 HTTPS



- CLOUDFRONT SUPPORTS HTTPS WITH TLS.
- USE ACM CERTIFICATES (IN US-EAST-1).
- ENFORCE HTTPS BY REDIRECTING HTTP TO HTTPS.
- SUPPORTED SSL/TLS PROTOCOLS: TLSv1.2 AND TLSv1.3.
- USE HSTS (HTTP STRICT TRANSPORT SECURITY) FOR ADDED SECURITY.

6 S3 WITH CLOUDFRONT



CLOUDFRONT DISTRIBUTION



USERS (EDGE LOCATIONS)

- STORE STATIC CONTENT IN AN S3 BUCKET.
- CREATE A CLOUDFRONT DISTRIBUTION WITH THE S3 BUCKET AS ORIGIN.
- CLOUDFRONT SERVES CONTENT FROM EDGE LOCATIONS.
- BLOCK PUBLIC ACCESS TO S3 BUCKET.
- USE ORIGIN ACCESS CONTROL (OAC) OR ORIGIN ACCESS IDENTITY (OAI) TO SECURE CONTENT.

7 STALE-CONTENT SCENARIOS

SCENARIO	POSSIBLE CAUSE	WHAT TO CHECK	HOW TO FIX
 OLD CONTENT STILL DISPLAYED AFTER UPDATE	• HIGH TTL SETTINGS • CONTENT CACHED AT EDGE LOCATIONS	• CHECK CACHE BEHAVIOR (TTL) • CHECK RESPONSE HEADERS (CACHE-CONTROL, EXPIRES) • CHECK WHEN CONTENT WAS LAST MODIFIED	• REDUCE TTL (MIN, DEFAULT, MAX) • INVALIDATE CLOUDFRONT CACHE • USE VERSIONED FILE NAMES (ETAGS)
 CHANGES TAKE TOO LONG TO REFLECT	• LONG TTL • NO CACHE INVALIDATION • BROWSER CACHING	• CHECK TTL SETTINGS • CHECK CACHE-CONTROL HEADERS • CHECK BROWSER CACHE	• LOWER TTL • CREATE INVALIDATION • CLEAR BROWSER CACHE OR USE HARD REFRESH
 PARTIAL STALE CONTENT	• SOME FILES UPDATED, OTHERS NOT • DIFFERENT TTL PER PATH	• CHECK INDIVIDUAL OBJECT LAST MODIFIED TIME • CHECK PATH-BASED CACHE BEHAVIOR	• INVALIDATE SPECIFIC PATHS • USE CONSISTENT TTL • UPDATE ALL RELATED FILES
 STALE CONTENT AFTER INVALIDATION	• INVALIDATION STILL IN PROGRESS • BROWSER CACHE • INTERMEDIATE PROXY CACHE	• CHECK INVALIDATION STATUS • CHECK CACHE-CONTROL HEADERS • CHECK BROWSER / PROXY CACHE	• WAIT FOR INVALIDATION TO COMPLETE • USE HARD REFRESH • PURGE PROXY / ISP CACHE

CLOUDFRONT BEST PRACTICES



USE CLOUDFRONT TO SPEED UP GLOBAL CONTENT DELIVERY.



USE OAC OR OAI TO SECURE S3 ORIGINS.



OPTIMIZE TTL BASED ON CONTENT UPDATE FREQUENCY.



ENFORCE HTTPS AND USE STRONG TLS SETTINGS.



MONITOR WITH CLOUDWATCH AND LOGS.

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



13. MONITORING WITH CLOUDWATCH



OBSERVE. ALERT. INVESTIGATE. RESOLVE.

1 METRICS



- NUMERIC DATA POINTS COLLECTED OVER TIME.
- AWS/NATIVE METRICS AND CUSTOM METRICS.
- NAMESPACE + METRIC NAME + DIMENSIONS.
- HIGH RESOLUTION: 1-SECOND OR 1-MINUTE GRANULARITY.
- USE METRICS TO UNDERSTAND HEALTH, PERFORMANCE, AND CAPACITY.

2 LOGS



- CENTRALIZE, STORE, AND SEARCH LOG DATA.
- SOURCES: EC2, ALB, VPC FLOW LOGS, APPLICATIONS, CONTAINERS, LAMBDA.
- LOG GROUPS CONTAIN LOG STREAMS.
- FILTER, SEARCH, AND ANALYZE LOGS.
- RETENTION SETTINGS CONTROL COST.

3 ALARMS



- MONITOR METRICS AND TAKE ACTION WHEN THRESHOLDS ARE BREACHED.
- STATES: OK, ALARM, INSUFFICIENT DATA.
- ACTIONS: SNS NOTIFICATION, AUTO SCALING, LAMBDA, SSM OPS ITEMS.
- SUPPORTS ANOMALY DETECTION.

4 DASHBOARDS



- VISUALIZE METRICS, LOGS, AND ALARMS.
- CREATE CUSTOM DASHBOARDS.
- ADD WIDGETS: LINE, BAR, NUMBER, TEXT, LOGS, ALARMS, ETC.
- SHARE DASHBOARDS ACROSS TEAMS.
- REAL-TIME INSIGHTS AT A GLANCE.

5 EC2 MONITORING



- CPU UTILIZATION, NETWORK IN/OUT, DISK I/O, STATUS CHECKS.
- MEMORY, DISK SPACE VIA AGENT (CW AGENT).
- INSTANCE STATUS CHECK (SYSTEM STATUS).
- INSTANCE REACHABILITY CHECK.
- DETAILED MONITORING ENABLED FOR 1-MINUTE METRICS.

6 APPLICATION MONITORING



- COLLECT APPLICATION LOGS AND CUSTOM METRICS (CLOUDWATCH AGENT, SDK, CONTAINERS).
- MONITOR ALB, RDS, LAMBDA, API GATEWAY, DYNAMODB, S3, ETC.
- END-TO-END VISIBILITY OF REQUESTS AND ERRORS.
- TRACK LATENCY, THROUGHPUT, AND FAILURE RATES.
- USE TRACES WITH X-RAY FOR DEEP INSIGHTS.

7 INVESTIGATING A PRODUCTION ALERT



- IDENTIFY THE ALERT AND IMPACT.
- CHECK RELATED METRICS AND ALARM HISTORY.
- REVIEW LOGS AROUND THE TIME OF THE ALERT.
- CORRELATE EVENTS ACROSS SERVICES.
- IDENTIFY ROOT CAUSE.
- TAKE CORRECTIVE ACTION AND VERIFY.
- UPDATE ALARMS AND DOCUMENT LESSONS.

8 INVESTIGATING A PRODUCTION ALERT – STEP BY STEP WORKFLOW

STEP	WHAT TO DO	CLOUDWATCH FEATURE	KEY QUESTIONS	OUTCOME
1	IDENTIFY THE ALERT	ALARMS	• WHICH ALARM TRIGGERED? • WHAT METRIC BREACHED THE THRESHOLD? • WHEN DID IT START?	UNDERSTAND WHAT HAPPENED AND WHEN
2	CHECK METRICS	METRICS	• HOW IS THE METRIC TRENDING? • IS THE ISSUE SPIKING OR SUSTAINED? • ANY CORRELATED METRICS?	CONFIRM IMPACT AND SCOPE
3	REVIEW LOGS	LOGS INSIGHTS	• WHAT ERRORS APPEAR AROUND THE ALERT TIME? • ANY EXCEPTIONS, TIMEOUTS, OR 5XX?	FIND CLUES AND ERROR DETAILS
4	CORRELATE EVENTS	DASHBOARDS + METRICS	• DID ANOTHER SERVICE/COMPONENT CHANGE AT THE SAME TIME? • ANY DEPLOYMENTS OR CONFIG CHANGES?	FIND RELATED CAUSE OR TRIGGER
5	IDENTIFY ROOT CAUSE	METRICS + LOGS + EVENTS	• WHAT EXACTLY CAUSED THE ISSUE? • IS IT INFRASTRUCTURE, APPLICATION, DEPENDENCY, OR TRAFFIC?	CONFIRM ROOT CAUSE
6	TAKE ACTION & VERIFY	ALARMS + METRICS + LOGS	• HOW WILL WE FIX IT? • IS THE METRIC BACK TO NORMAL? • ARE USERS IMPACTED ANYMORE?	RESOLVE AND VALIDATE THE FIX
7	IMPROVE & DOCUMENT	DASHBOARDS + ALARMS + LOGS	• CAN WE PREVENT OR DETECT FASTER? • DO WE NEED NEW ALARMS/DASHBOARDS? • DOCUMENT WHAT HAPPENED.	PREVENT RECURRENCE AND IMPROVE

CLOUDWATCH BEST PRACTICES



MONITOR WHAT MATTERS



ALARM EARLY, NOT LATE



USE DASHBOARDS FOR VISIBILITY



CENTRALIZE & RETAIN LOGS



CONTROL COSTS WITH RETENTION & METRICS



REVIEW & IMPROVE ALARMS REGULARLY

VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



14.

AWS LOGGING AND AUDITING

EVERY ACTION. EVERY CHANGE. EVERY TIME.

1 CLOUDTRAIL

Auditing API activity



- Records API calls made in your AWS account.
- Who made the request.
- When it happened.
- From where (IP address).
- What was requested.
- Success or failure.

Use for:

- Security auditing
- Compliance
- Investigating API activity
- Finding who changed a resource

2 CLOUDWATCH LOGS

Application and system logs



- Collects, stores, and searches log data.
- Logs from EC2, Lambda, containers, applications, and services.
- Centralized log storage.
- Real-time log monitoring.
- Log retention and metric filters.

Use for:

- Application troubleshooting
- System and access logs
- Search and analyze logs
- Create metrics and alarms from logs

3 VPC FLOW LOGS

Network traffic logging



- Captures IP traffic going to and from network interfaces in your VPC.
- Source/Destination IP addresses.
- Ports and protocols.
- Allow or reject.
- Bytes and packets.

Use for:

- Network security analysis
- Troubleshooting connectivity
- Detecting unusual traffic
- Compliance and forensics

4 WHAT EACH SERVICE RECORDS

FEATURE	CLOUDTRAIL	CLOUDWATCH LOGS	VPC FLOW LOGS
What it records	API activity and account events	Log data from resources and applications	Network traffic in your VPC
Who/What	Users, roles, AWS services	EC2, Lambda, containers, apps, services, OS	Network interfaces (ENI)
Details captured	Who, When, Where, What, Success/Failure	Log messages, events, errors, metrics	Source/Dest IP, ports, protocol, action, bytes, packets
Granularity	API call level	Log event level	Network connection level
Retention	S3 bucket (you choose)	1 day to 10 years (configurable)	S3 / CloudWatch Logs (configurable)
Best for	Auditing, compliance, change tracking	Troubleshooting, monitoring, analytics	Network security, connectivity, threat detection

5 FINDING WHO CHANGED AN AWS RESOURCE (EXAMPLE)



Open CloudTrail in AWS Console.



Go to Event history or Lake (advanced).



Filter by:

- Event name
- Resource name
- Time range
- User/Role



Review the event details:

- Who made the change
- What was changed
- When it happened
- Source IP address



Download the event or save for audit.

EXAMPLE: Find who stopped an EC2 instance

Event name	StopInstances
User name	john.doe
Event time	2025-07-19T10:15:22Z
Source IP	203.0.113.45
Resource	i-0abc123def4567890
Response	Success

Additional details (JSON)

```
{
  "eventSource": "ec2.amazonaws.com",
  "eventName": "StopInstances",
  "awsRegion": "us-east-1",
  "userIdentity": {
    "type": "IAMUser",
    "userName": "john.doe"
  },
  "requestParameters": {
    "instancesSet": {
      "items": ["i-0abc123def4567890"]
    }
  }
}
```

6 BASIC INCIDENT INVESTIGATION WORKFLOW

Identify the Issue
Alert, ticket, or user report.Gather Evidence
Check CloudTrail, CloudWatch Logs, Flow Logs, Metrics, and Alarms.Check CloudTrail
Find related API calls and changes around the time.Review VPC Flow Logs
Analyze network traffic for blocked or suspicious activity.Correlate Findings
Connect the dots across all logs and metrics.Document & Resolve
Document root cause, actions taken, and lessons learned.ENABLE CLOUDTRAIL
IN ALL REGIONS.STORE CLOUDTRAIL LOGS
IN A DEDICATED S3 BUCKET
WITH VERSIONING.

BEST PRACTICES

USE CLOUDWATCH LOG
GROUPS WITH APPROPRIATE
RETENTION.ENABLE VPC FLOW LOGS
FOR IMPORTANT VPCS
AND SUBNETS.CREATE ALARMS FROM
METRICS AND
LOG PATTERNS.

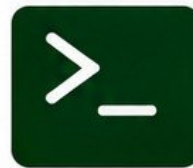
VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



AWS CLI AND OPERATIONAL SKILLS

AUTOMATE. INSPECT. TROUBLESHOOT. OPERATE.



1 AWS CLI CONFIGURATION

Install AWS CLI

```
# Check version
aws --version
```

Configure AWS CLI

```
aws configure
AWS Access Key ID [None]: AKIAXXXXXXXXXXXX
AWS Secret Access Key [None]: *****
Default region name [None]: us-east-1
Default output format [None]: json
```



This creates/updates credentials in:
~/.aws/credentials and ~/.aws/config

2 PROFILES

Use multiple profiles for different accounts or roles.

Create a profile

```
aws configure --profile dev
aws configure --profile prod
```

Use a profile

```
aws s3 ls --profile dev
```



Profiles are stored in:
~/.aws/credentials
and ~/.aws/config

3 CREDENTIALS

- Access Key ID & Secret Access Key
- IAM User or IAM Role credentials
- Stored securely in ~/.aws/credentials
- Use IAM Roles on EC2/ECS/Lambda whenever possible (no keys needed)



BEST PRACTICE

Never hardcode keys.
Use IAM roles and least privilege permissions.

4 COMMON COMMANDS

S3 COMMANDS

```
# List buckets
aws s3 ls

# List objects in bucket
aws s3 ls s3://my-bucket

# Copy file to S3
aws s3 cp file.txt s3://my-bucket/

# Sync folder to S3
aws s3 sync ./my-folder s3://my-bucket/
```

EC2 COMMANDS

```
# List instances
aws ec2 describe-instances

# List running instances
aws ec2 describe-instances \
  --filters Name=instance-state-name, \
  Values=running

# Start instance
aws ec2 start-instances \
  --instance-ids i-1234567890abcdef0

# Stop instance
aws ec2 stop-instances \
  --instance-ids i-1234567890abcdef0
```

IAM COMMANDS

```
# List users
aws iam list-users

# List roles
aws iam list-roles

# Get user details
aws iam get-user \
  --user-name john.doe

# List policies
aws iam list-policies \
  --scope Local
```

GENERAL COMMANDS

```
# Help
aws help

# List services
aws services list

# Get caller identity
aws sts get-caller-identity

# Set region
aws configure set region us-east-1

# Set output format
aws configure set output json
```

5 RESOURCE INSPECTION

USE CASE	EXAMPLE COMMAND
Check EC2 instance details	<code>aws ec2 describe-instances \</code> <code>--instance-ids i-1234567890abcdef0</code>
Check EBS volumes	<code>aws ec2 describe-volumes \</code> <code>--filters Name=status,Values=available</code>
Check Security Groups	<code>aws ec2 describe-security-groups \</code> <code>--group-ids sg-12345678</code>
Check VPCs	<code>aws ec2 describe-vpcs</code>
Check ALBs	<code>aws elbv2 describe-load-balancers</code>
Check RDS instances	<code>aws rds describe-db-instances</code>
Check S3 bucket details	<code>aws s3api head-bucket \</code> <code>--bucket my-bucket</code>
Check CloudWatch Alarms	<code>aws cloudwatch describe-alarms</code>

6 TROUBLESHOOTING WITH THE CLI

Check API errors	<code>aws <service> <operation> --debug</code> # Shows request/response details
Test connectivity	<code>aws sts get-caller-identity</code> # Confirms credentials & permissions
Check permissions	<code>aws iam simulate-principal-policy \</code> <code>--policy-source-arn arn:aws:iam::123456789012:user/john \</code> <code>--action-names s3:ListBucket</code>
Dry run operations	<code>aws s3 sync ./folder s3://bucket/ --dryrun</code> # See what would be changed
Filter and search	<code>aws ec2 describe-instances \</code> <code>--filters Name=tag:Name,Values=web \</code> <code>--query "Reservations[].Instances[].InstanceId" \</code> <code>--output table</code>
Retry on failure	<code>aws s3 cp largefile s3://bucket/ --cli-retry-mode adaptive</code> # Better handling of throttling

7 READING COMMAND OUTPUT

OUTPUT FORMATS

- json (default)
- table (human readable)
- text (raw text)
- yaml / yaml-stream

Set default output format

```
aws configure set output table
```

EXAMPLE OUTPUT (TABLE)

```
aws ec2 describe-instances --query \
  "Reservations[].Instances[].State.StateName" --output table
```

ID	STATE
i-1234567890abcdef0	running
i-0fedcba9876543210	stopped
i-0a1b2c3d4e5f6a7b8	running

USE JMESPATH QUERIES

Extract only what you need.
Example: Get all instance IDs

```
aws ec2 describe-instances \
  --query "Reservations[].Instances[].InstanceId" \
  --output text
```

Example: Count running instances

```
aws ec2 describe-instances \
  --filters Name=instance-state-name,Values=running \
  --query "length(Reservations[].Instances[])"
```

BEST PRACTICES



USE IAM ROLES
INSTEAD OF
ACCESS KEYS



ROTATE ACCESS
KEYS REGULARLY
IF YOU MUST USE



USE LEAST
PRIVILEGE
PERMISSIONS



USE PROFILES
FOR MULTIPLE
ACCOUNTS



AUTOMATE WITH
CLI + SCRIPTS
+ PIPELINES



LOG AND MONITOR
CLI ACTIVITY WITH
CLOUDTRAIL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta

VERIQT SOCIAL



SERVERLESS AWS FUNDAMENTALS

BUILD LESS. SCALE MORE. PAY ONLY FOR WHAT YOU USE.



1 LAMBDA



- Run code without managing servers.
- Supports multiple runtimes (Python, Node.js, Java, Go, etc.).
- Automatic scaling based on requests.
- Pay per request and compute time.

2 API GATEWAY



- Create, publish, and secure APIs.
- Integrates with Lambda and other AWS services.
- Handles authentication, throttling, and quotas.
- Supports REST APIs and HTTP APIs.

3 EVENT-DRIVEN EXECUTION



- Functions run in response to events.
- Event sources include: API Gateway, S3, SNS, SQS, DynamoDB, EventBridge, CloudWatch Events, etc.
- Perfect for reactive and scalable workloads.

4 PERMISSIONS



- Use IAM roles for Lambda execution.
- Grant least privilege permissions.
- Control who can invoke the function.
- Use resource-based policies for API Gateway and Lambda.

5 CLOUDWATCH LOGS



- Lambda automatically sends logs to CloudWatch Logs.
- View function logs in real time.
- Filter, search, and analyze log data.
- Set log retention to manage storage and cost.

6 TIMEOUTS



- Configure function timeout (1 second to 15 minutes).
- Function stops when timeout is reached.
- Choose the right timeout to balance performance and cost.
- Monitor duration and adjust as needed.

7 BASIC SERVERLESS TROUBLESHOOTING



- Check CloudWatch Logs for errors.
- Verify IAM permissions.
- Check timeout settings.
- Validate function code and dependencies.
- Use X-Ray for tracing and debugging.

8 TYPICAL SERVERLESS ARCHITECTURE (EXAMPLE)



CLIENT



API GATEWAY

LAMBDA
FUNCTION

S3 BUCKET

CLOUDWATCH
LOGS

User sends request to API Gateway.

API Gateway forwards the request to Lambda.

Lambda processes the request and performs the business logic.

Lambda reads from or writes to S3 as needed.

Lambda logs are stored in CloudWatch Logs for monitoring.

9 BASIC SERVERLESS TROUBLESHOOTING CHECKLIST

STEP	WHAT TO CHECK	HOW TO CHECK	WHAT TO LOOK FOR
1	Function Logs	CloudWatch Logs → Log Groups → /aws/lambda/your-function	Errors, exceptions, stack traces, and custom log messages
2	Permissions	IAM Console → Roles → Lambda execution role	Missing permissions, access denied errors, incorrect policies
3	Timeouts	Lambda Console → Configuration → General Configuration	Function timeout too low, function timing out
4	Environment Variables	Lambda Console → Configuration → Environment variables	Missing or incorrect environment variables
5	Dependencies	Check deployment package or layers	Missing libraries or incorrect versions
6	API Gateway Integration	API Gateway Console → Stages → Logs / Test	Integration errors, 4XX/5XX responses, mapping issues
7	Throttling / Concurrency	CloudWatch Metrics → Lambda → Throttles, ConcurrentExecutions	Throttling errors, account concurrency limits reached

BEST PRACTICES



KEEP FUNCTIONS SMALL AND FOCUSED



USE IAM ROLES AND LEAST PRIVILEGE PERMISSIONS



MONITOR WITH CLOUDWATCH LOGS & METRICS



SET APPROPRIATE TIMEOUTS AND MEMORY



HANDLE ERRORS AND LOG MEANINGFULLY

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



AWS SECURITY FUNDAMENTALS

SECURE BY DESIGN. LEAST PRIVILEGE. ALWAYS VERIFY.



1 SHARED RESPONSIBILITY MODEL



AWS (RESPONSIBLE FOR)

- Global infrastructure
- Regions, AZs, Edge locations
- Hardware, networking, virtualization
- Managed services
- Physical security

CUSTOMER (RESPONSIBLE FOR)

- Data and content
- IAM users and permissions
- OS, patching, and applications
- Network configuration
- Security groups and NACLs
- Monitoring and logging

2 ENCRYPTION



- Encrypt data at rest using AWS services.
- Encrypt data in transit using TLS.
- Use strong encryption algorithms (AES-256).
- Enable encryption by default where possible.
- Control access to encryption keys.

EXAMPLES

- S3 SSE-S3, SSE-KMS
- EBS Encryption
- RDS Encryption
- TLS for APIs and Load Balancers

3 KMS BASICS



- AWS Key Management Service creates and manages keys.
- Use customer managed keys (CMKs) for more control.
- Keys never leave AWS KMS.
- Control who can use keys with IAM policies.
- Enable key rotation for security.

KEY TYPES

- AWS Managed Keys
- Customer Managed Keys
- AWS Owned Keys

4 SECRETS MANAGEMENT



- Store secrets securely using AWS Secrets Manager or SSM Parameter Store.
- Never hardcode secrets in code.
- Use IAM permissions to restrict access.
- Rotate secrets regularly.

EXAMPLES

- Database passwords
- API keys
- OAuth tokens
- Private keys

5 IAM SECURITY



- Use IAM users for humans, not root.
- Use IAM roles for applications and services.
- Follow least privilege principle.
- Use groups to manage permissions.
- Enable MFA for all users.
- Review and remove unused access.
- Never share access keys.

BEST PRACTICES

- Least privilege
- MFA enabled
- Strong password policy
- Regular access review

6 NETWORK SECURITY



- Design VPC with public and private subnets.
- Use Security Groups (stateful).
- Use NACLs (stateless) for subnet-level control.
- Avoid exposing services to the internet.
- Use VPC Endpoints to access AWS services privately.
- Enable AWS WAF for web applications.
- Use TLS for all external communication.

KEY CONTROLS

- Security Groups
- NACLs
- VPC Endpoints
- AWS WAF
- TLS/HTTPS
- Private Subnets

7 COMMON JUNIOR-LEVEL SECURITY MISTAKES



- Using the root account for daily tasks.
- Overly permissive IAM policies (*:*).
- Hardcoding secrets in code or config.
- Leaving S3 buckets public.
- Exposing databases to the internet.
- Disabling or ignoring security groups.
- Not enabling encryption.
- Not reviewing IAM access regularly.
- Storing keys in code or local files.
- Ignoring CloudTrail and CloudWatch security alerts.

8 SECURITY SERVICES AT A GLANCE

SERVICE	PURPOSE	KEY BENEFITS	JUNIOR USE CASES
IAM	Manage access to AWS resources.	Least privilege access, MFA, roles, policies.	Create users, roles, groups, and permissions.
KMS	Manage encryption keys.	Centralized key management, audit, rotation.	Encrypt S3, EBS, RDS, Secrets, use CMKs.
SECRETS MANAGER	Store and rotate secrets securely.	Automatic rotation, encryption, fine-grained access.	Store DB passwords, API keys, rotate automatically.
CLOUDTRAIL	Log and monitor AWS API activity.	Audit trail, compliance, investigation.	Track who did what, when, and from where.
CLOUDWATCH	Monitor resources and set alarms.	Metrics, logs, alarms, dashboards.	Set alarms, monitor errors, review logs.
AWS WAF	Protect web apps from threats.	Block common attacks, IP filtering, rate limiting.	Protect APIs, ALB, CloudFront from bad traffic.

9 GOLDEN SECURITY PRINCIPLES



LEAST PRIVILEGE



ENCRYPT EVERYTHING



MONITOR ALWAYS



AUDIT EVERY ACTION



AUTOMATE SECURITY



EDUCATE CONTINUOUSLY



DEFENSE IN DEPTH

BEST PRACTICES



ENABLE MFA EVERYWHERE



USE ROLES INSTEAD OF USERS WHEN POSSIBLE



ENCRYPT DATA AT REST AND IN TRANSIT



REVIEW ACCESS REGULARLY AND REMOVE UNUSED



MONITOR AND ALERT ON SUSPICIOUS ACTIVITY



KEEP SYSTEMS PATCHED AND UP TO DATE

VERIQTA SOCIAL



YouTube
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Instagram
veriqta



Telegram
veriqta



VERIQTA ENGINEERING EXCELLENCE ARCHITECTURE INTERVIEW SCENARIOS



VERIQTA



DESIGN A BASIC WEB APPLICATION

SCENARIO

Design a highly available, scalable, and secure web application on AWS.

HIGH AVAILABILITY



No single point of failure.

SCALABILITY



Handle traffic spikes automatically.

SECURITY



Least privilege and network isolation.

PERFORMANCE



Fast, reliable, and responsive application.

COST EFFECTIVE



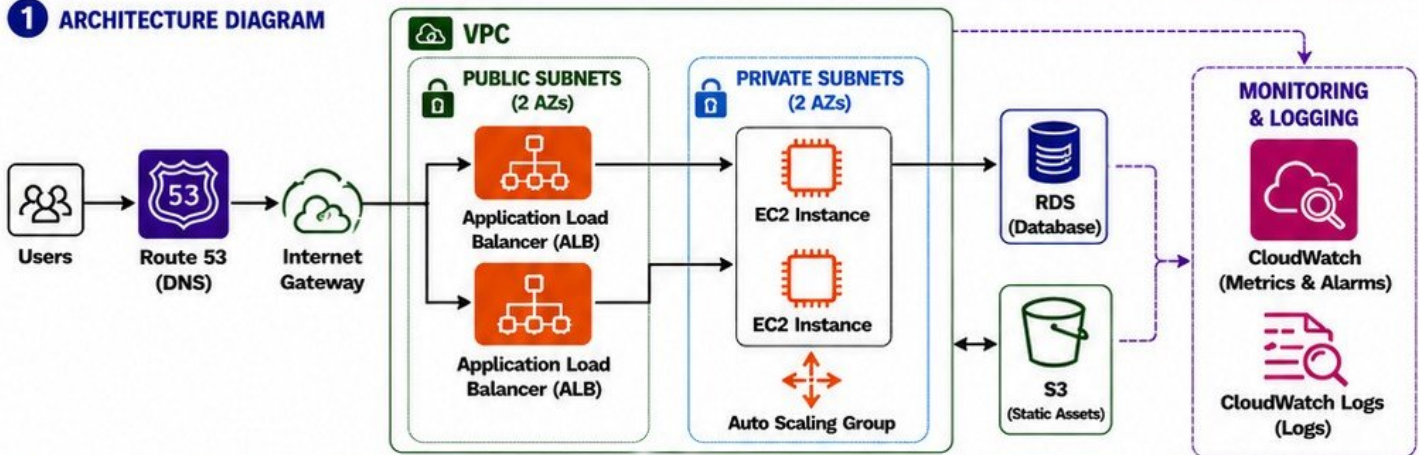
Pay for what you use.

MONITORING



Full visibility and alerting.

1 ARCHITECTURE DIAGRAM



2 COMPONENTS OVERVIEW

	ROUTE 53	Domain registration and DNS routing to the Application Load Balancer. Provides high availability and global DNS.
	INTERNET GATEWAY	Allows communication between the internet and your VPC.
	APPLICATION LOAD BALANCER (ALB)	Distributes incoming HTTP/HTTPS traffic across EC2 instances in multiple Availability Zones.
	EC2 INSTANCES	Runs the web application. Deployed in private subnets for security.
	AUTO SCALING GROUP	Automatically adjusts the number of EC2 instances based on demand.
	RDS (DATABASE)	Managed relational database for persistent data (Multi-AZ for HA).
	S3 (STATIC ASSETS)	Stores static files like images, CSS, JS. Accessed securely by the application.
	CLOUDWATCH	Monitors metrics, sets alarms, and provides dashboards.
	CLOUDWATCH LOGS	Centralized logging for application, system, and access logs.

3 ARCHITECTURAL DECISIONS EXPLAINED

	ROUTE 53	Reliable DNS with health checks and failover routing.
	ALB IN PUBLIC SUBNETS	Handles SSL termination and distributes traffic AZs.
	EC2 IN PRIVATE SUBNETS	Improves security by preventing direct internet access to instances.
	AUTO SCALING GROUP	Ensures the application scales automatically with traffic changes.
	RDS MULTI-AZ	Provides high availability and automatic failover for the database.
	S3 FOR STATIC CONTENT	Durable, scalable, and cost-effective storage for static assets.
	CLOUDWATCH MONITORING	Real-time visibility into performance, errors, and infrastructure health.
	CLOUDWATCH LOGS	Centralized logs for easier troubleshooting and auditing.
	SECURITY BEST PRACTICES	Least privilege IAM, security groups, NACLs, encryption in transit and at rest.

4 BENEFITS OF THIS ARCHITECTURE

	HIGH AVAILABILITY	Resources are deployed across multiple Availability Zones.
	SCALABLE	Handles traffic spikes automatically with Auto Scaling and ALB.
	SECURE	Private subnets, security groups, and least privilege access.
	RELIABLE	Managed services (RDS, S3, Route 53) provide built-in durability.
	COST-EFFECTIVE	Pay only for what you use with AWS managed services.
	OBSERVABLE	Full monitoring, metrics, and logs for faster issue detection and resolution.

5 INTERVIEW TIPS

- Always ask clarifying questions (traffic, availability, region, budget).
- Start with a high-level diagram, then add details.
- Explain why you choose each service.
- Consider security, scalability, cost, and operational excellence.
- Mention trade-offs and alternatives when relevant.

BEST PRACTICES TO REMEMBER

Follow least privilege principle.	Encrypt data in transit and at rest.	Keep systems patched and updated.	Monitor proactively and set alarms.	Backup regularly and test restore.	Optimize costs continuously.
-----------------------------------	--------------------------------------	-----------------------------------	-------------------------------------	------------------------------------	------------------------------

VERIQTA SOCIAL

YouTube veriqta	GitHub veriqta	LinkedIn veriqta	X veriqta	Instagram veriqta	Telegram veriqta
--------------------	-------------------	---------------------	--------------	----------------------	---------------------



19.

REAL-WORLD AWS
TROUBLESHOOTING INTERVIEW

INVESTIGATE SYSTEMATICALLY. THINK LIKE AN ENGINEER.



1 COMMON INTERVIEW TROUBLESHOOTING SCENARIOS

**EC2 IS UNREACHABLE**Instance down,
SG, NACL, route.**ALB TARGET IS UNHEALTHY**Target health,
logs, config.**APPLICATION CANNOT REACH RDS**Security group,
subnet, route.**S3 RETURNS ACCESSDENIED**Bucket policy,
IAM, ACL.**DNS POINTS TO THE WRONG ENDPOINT**Route 53 record,
TTL, cache.**CPU SUDDENLY SPIKES**Load, app,
leaks.**LAMBDA TIMES OUT**Timeout,
memory.

2 HOW TO INVESTIGATE SYSTEMATICALLY



1. OBSERVE

- Check alerts
- Read logs
- Confirm impact



2. VERIFY

- Check status
- Test service
- Reproduce



3. ISOLATE

- Narrow scope
- Check deps
- Find root cause



4. RESOLVE

- Apply fix
- Test again
- Confirm



5. DOCUMENT

- Record findings
- Update runbook
- Prevent repeat

3 TROUBLESHOOTING DEEP DIVE

ISSUE	WHAT TO CHECK	KEY COMMANDS / LOGS	LIKELY CAUSE	FIX
EC2 Unreachable	• Instance state • Security group • NACL • Route table	<pre>aws ec2 describe-instances aws ec2 describe-security-groups aws ec2 describe-network-acls</pre>	SG/NACL blocks, wrong route, instance stopped	Open SG/NACL, fix route, start instance
ALB Target Unhealthy	• Target health • App logs • Security group	<pre>aws elbv2 describe-target-health aws logs tail /var/log/nginx</pre>	App down, health check fail	Fix app, allow SG, update health path
Application Cannot Reach RDS	• SG (EC2 ↔ RDS) • Subnet routes • DB status	<pre>aws rds describe-db-instances mysql -h <endpoint> -u <user> -p</pre>	SG blocks, wrong subnet, DB down	Open SG, correct subnet, restart DB
S3 Returns AccessDenied	• Bucket policy • IAM role/user • Block public access	<pre>aws s3 ls aws sts get-caller-identity</pre>	IAM no permission, bucket policy	Grant IAM or fix bucket policy
DNS Points to Wrong Endpoint	• Route 53 records • TTL • Health checks	<pre>dig yourdomain.com aws route53 list-resource-record-sets</pre>	Old record, low TTL, wrong alias	Update record, set correct alias
CPU Suddenly Spikes	• Top processes • CloudWatch metrics • Logs	<pre>top htop aws cloudwatch get-metric-statistics</pre>	App leak, traffic spike, background job	Kill process, scale, optimize code
Lambda Times Out	• Duration metrics • Memory size • Logs	<pre>aws logs tail /aws/lambda/... aws lambda get-function-configuration</pre>	Too little memory, slow code	Increase memory, optimize code



KEY TROUBLESHOOTING RULES

- ✓ Check CloudWatch logs first.
- ✓ Validate IAM and security groups.
- ✓ Test connectivity end-to-end.
- ✓ Change one thing at a time.



FINAL RESULT:

- Find the real cause.
- Apply the right fix.
- Restore service fast.

VERIQTA SOCIAL

YouTube
veriqtaGitHub
veriqtaLinkedIn
veriqtaX
veriqtaInstagram
veriqtaTelegram
veriqta



JUNIOR AWS MOCK INTERVIEW

PREPARE. PRACTICE. PERFORM.



1 RAPID TECHNICAL QUESTIONS

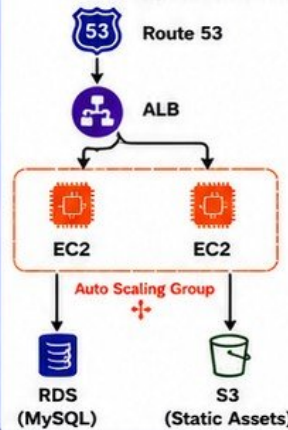
- What is EC2 and when would you use it?
- What is the difference between S3 and EBS?
- What is the difference between ALB and NLB?
- What is a VPC?
- What is a Security Group?
- What is an IAM Role?
- What is the difference between public and private subnets?
- What is Auto Scaling?
- What is CloudWatch used for?
- What is the default VPC?

2 SCENARIO QUESTIONS

- Your website is slow for users in another region. What could help?
- You need to host static files that must be publicly accessible. What service would you use?
- Your application must be highly available. What services would you combine?
- You registered a domain. How do you route traffic to your application?
- You need to run code in response to an S3 upload. How would you do that?

3 ARCHITECTURE CHALLENGE

Design a scalable, highly available web application on AWS.



Explain your choices:

- ✓ High Availability
- ✓ Scalability
- ✓ Security
- ✓ Cost Optimization
- ✓ Monitoring

4 TROUBLESHOOTING CHALLENGE

Your users cannot access the application.

1. Identify the Problem
2. Gather Evidence
3. Narrow Down
4. Fix and Verify

Layer	What to Check	Example Commands / Tools	Common Issues
DNS	Route 53 record, TTL, correct record type	<code>aws route53 list-resource-record-sets --hosted-zone-id ZONEID</code>	Wrong record, TTL caching, wrong alias
Network	Security Group, NACL, Subnet, Route Table, Internet Gateway	<code>aws ec2 describe-security-groups</code> <code>aws ec2 describe-route-tables</code>	SG blocks traffic, missing route, no IGW
Load Balancer	Target health, listener, listener rules	<code>aws elbv2 describe-target-health</code> <code>aws elbv2 describe-listeners</code>	Unhealthy targets, wrong listener, bad rule
EC2	Instance state, SG, app status, logs	<code>aws ec2 describe-instances</code> <code>tail -f /var/log/nginx/error.log</code>	Instance stopped, app down, SG blocks
Application	App logs, errors, configuration	<code>journalctl -u app -f</code> Check CloudWatch Logs	Code error, bad config, missing env variable
Database	RDS status, SG, endpoint, credentials	<code>aws rds describe-db-instances</code>	DB down, SG blocks, wrong endpoint

5 SECURITY QUESTIONS

- What is the AWS Shared Responsibility Model?
- How would you secure an S3 bucket?
- How would you prevent an IAM user from deleting resources?
- What is the difference between IAM User and IAM Role?
- How would you secure data in transit and at rest?
- How would you manage secrets in an application?
- How would you allow EC2 to access S3 securely?

6 "WHAT WOULD YOU CHECK FIRST?" QUESTIONS

- ✓ An EC2 instance is unreachable. What would you check first?
- ✓ An ALB target is unhealthy. What would you check first?
- ✓ S3 returns AccessDenied. What would you check first?
- ✓ High CPU on EC2. What would you check first?
- ✓ Lambda function fails. What would you check first?
- ✓ Application cannot reach RDS. What would you check first?
- ✓ DNS points to the wrong endpoint. What would you check first?

7 STRONG ANSWERS vs WEAK ANSWERS

STRONG ANSWERS (DO THIS)

- ✓ Give clear, concise, and structured answers.
- ✓ Explain your thought process.
- ✓ Mention multiple valid approaches when possible.
- ✓ Use AWS best practices in your answers.
- ✓ Communicate clearly and with confidence.
- ✓ Ask clarifying questions when needed.

WEAK ANSWERS (AVOID THIS)

- ✗ Give one-word or short vague answers.
- ✗ Jump to solutions without understanding the problem.
- ✗ Ignore cost, security, or scalability.
- ✗ Say "I don't know" without explaining how you would find out.
- ✗ Forget to explain trade-offs.
- ✗ Sound unsure or not confident.

8 FINAL INTERVIEW READINESS CHECKLIST

- ✓ Understand core AWS services deeply
- ✓ Practice real-world scenarios
- ✓ Know how to troubleshoot systematically
- ✓ Understand security best practices
- ✓ Explain architecture decisions clearly
- ✓ Use AWS CLI and Console confidently
- ✓ Communicate clearly and confidently
- ✓ Stay calm and think out loud

AWS INTERVIEW SUCCESS FORMULA

